

第1章 编译概述

1.1 知识要点

本章简要介绍编译程序的各个组成部分,对编译各逻辑阶段的内容进行概述。多数概念在以后的各章中有详细介绍。本章要求掌握以下内容:

- (1) 源语言、目标语言、实现语言。
- (2) 翻译程序、解释程序、编译程序。
- (3) 编译程序的各个逻辑阶段及其主要功能。
- (4) 编译的前端、后端、遍。
- (5) 编译程序的伙伴工具。

1.2 习题解析及参考答案

1.1 高级程序设计语言有哪两种翻译方式?它们的特点分别是什么?

解答:

对高级程序设计语言的翻译有解释和编译两种方式。解释程序直接将源程序翻译成执行结果,编译程序则是把源程序翻译成另外一种形式,如汇编语言程序或机器语言程序。

解释程序对源程序进行解释执行,其特点是:边解释边执行,不生成目标程序,每次执行都需要对源程序重新进行分析解释。

编译程序将源程序转换成目标程序,目标程序在机器上执行。源程序的编译和执行在不同的时间进行,且目标程序可以重复执行而不需要重新编译。

1.2 典型的编译程序可以划分为哪几个主要部分?各部分的主要功能是什么?

解答:

典型的编译程序划分为词法分析、语法分析、语义分析、中间代码生成、代码优化以及目标代码生成6个部分。各部分的主要作用如下。

(1) 词法分析:扫描字符串表示的源程序,根据词法规则识别出具有独立意义的单词符号,输出记号流。

(2) 语法分析:分析程序的逻辑结构,根据源语言的语法规则把记号流按语法结构按层次分组,以形成语法短语。

(3) 语义分析:对语法成分的类型、含义进行检查,以保证程序各部分能够有机地结合在一起,并为以后生成目标代码收集必要的信息,如类型、目标地址等。

(4) 中间代码生成:将源程序转换为一种中间表示形式,以便于进一步转换成目标代码。

(5) 目标代码生成: 将中间形式代码转换成目标代码。

(6) 代码优化: 包含中间代码优化和目标代码优化两类。对中间代码或目标代码进行改进以获得高效的代码, 即使优化后的代码占用的空间少、运行速度快。

1.3 编译程序的设计与实现过程涉及哪些方面的知识? 它们分别对编译程序有什么影响?

解答:

编译程序的设计与实现涉及以下几方面的知识。

- 程序设计语言: 源语言、目标语言以及编译程序的实现语言。
- 形式语言与自动机理论: 这是编译程序的理论基础, 语言的结构可以用文法进行形式化描述, 利用文法与自动机之间的等价关系, 可基于自动机理论生成词法和语法分析程序。
- 计算机体系结构: 决定了目标语言, 影响目标代码生成。
- 数据结构、算法分析与设计: 由于程序设计语言中都有数据结构、控制结构等, 对源语言和目标语言中结构的理解、它们之间的映射关系、如何编写编译程序实现从源语言到目标语言的转换等, 都离不开数据结构、算法分析与设计相关的知识。
- 操作系统: 操作系统是编译程序工作的基础, 编译程序的实现要借助于底层操作系统提供的功能, 如存储空间分配和管理。
- 软件工程: 编译程序是一个复杂的系统程序, 需要在软件工程的思想指导下进行规范的设计开发。

1.4 编译程序有哪些伙伴工具? 它们的作用是什么?

解答:

编译程序的伙伴工具主要有预处理器、汇编程序、连接装配程序。它们的作用如下。

- 预处理器: 对源程序进行处理, 产生编译程序的输入。预处理器主要完成宏处理、文件包含、语言扩充等功能。
- 汇编程序: 有些编译程序产生汇编语言的目标代码, 此时就需要由汇编程序对目标代码作进一步处理, 生成可重定位的机器代码。
- 连接装配程序: 完成连接和装配任务。所谓连接, 即把几个可重定位的机器代码文件连接成一个可执行的程序, 这些文件可以是分别编译或汇编得到的, 也可以是由系统提供的、对任何需要它们的程序而言都可用的子程序组成的库文件。所谓装配, 即读入可重定位的机器代码, 修改需重定位的地址, 把修改后的指令和数据放在内存中适当的地方或形成可执行文件。

1.5 解释下列名词: 翻译程序、编译程序、汇编程序、解释程序、编译程序的遍、编译程序的前端、编译程序的后端。

解答:

翻译程序是把用某种语言书写的程序翻译成计算机能够执行的表示形式或直接执行这个程序的程序。

编译程序是翻译程序的一种, 它把高级语言源程序转换成目标机器的机器语言或汇编语言程序。

汇编程序是编译程序的一种,它把汇编语言程序转换成目标机器的机器语言程序。

解释程序是翻译程序的一种,它直接对源程序进行解释执行。

编译程序的“遍”是指对源程序或其中间表示形式从头到尾扫描一次,并在扫描过程中作相关的加工处理,生成新的中间表示形式或目标程序。

编译程序的前端主要由与源语言有关而与目标机器无关的部分组成,通常包括词法分析、语法分析、语义分析、中间代码生成、符号表的建立以及与机器无关的代码优化工作,当然,前端也包括相应的错误处理工作和符号表操作。

编译程序的后端由编译程序中与目标机器有关的部分组成,一般来讲,这些部分与源语言无关而仅仅依赖于中间语言。后端包括目标代码的生成、与机器有关的代码优化以及相应的错误处理和符号表操作。

1.6 将编译程序划分成前端和后端的目的是什么?

解答:

将编译程序划分成前端和后端的目的是便于编译程序的移植和构造。例如,重写某个编译程序的后端,可以将该源语言的编译程序移植到另一种机器上,这种方法已经得到了普遍应用。重写某编译程序的前端,使之能把一种新的源语言编译成同一种中间语言,利用原编译程序的后端,从而可构造出新源语言在此机器上的编译程序。

1.7 请指出下面的错误可在编译的哪个阶段发现。

- 关键字拼写错误,如把 while 误写为 whlie。
- 缺少运算对象,如本应为 $a=3+b;$,却误写为 $a=3+;$ 。
- 实参与形参的类型不一致。
- 引用的变量没有定义。
- 数组下标越界。
- 本应为常数,但在数中出现了非数字字符,如误把 2.5 写成 2m5。

解答:

关键字拼写错误可以在语法分析阶段发现。例如,把 while 误写为 whlie,把 for 误写为 fro 等,词法分析程序会把它们识别为用户定义的标识符,只有到了语法分析阶段,在把记号组合成语法成分时才会发现这类问题。

缺少运算对象的错误可以在语法分析阶段发现。例如,源程序中出现了表达式 $a+*b$,词法分析程序可以识别出每个单词,依次为标识符 a、加号、乘号、标识符 b;而在语法分析阶段,在把它们组合成表达式过程中,根据表达式的定义,会发现两个运算符之间缺少运算对象。同样,对于源程序中出现的语句 $a=3+;$,词法分析程序依次识别出标识符 a、赋值号、常数 3、加号、分号,而在分析表达式结构时会发现缺少运算对象。

实参与形参的类型不一致的错误可以在语义分析阶段进行类型检查时发现。例如,某 C 语言程序中有如下的变量声明语句和函数定义:

```
int a, b;
string s;
int fx(int i, int j) {...}
```

在程序体中调用该函数的语句是

```
a=fx(b, s);
```

则在语义分析阶段,要检查实参和形参的个数是否相同,以及实参与相应形参的类型是否相同或相容,若发现不一致,则报告错误。

引用的变量没有定义的错误可以在语义分析阶段进行作用域检查时发现。例如,对于C语言赋值语句 $x=a+4$; ,其中的名字 x 和 a 要么是在该赋值语句所在的函数中声明的局部变量,要么是全局变量。在分析该赋值语句时,首先在局部变量表(即函数的子符号表)中查找,若没有找到,则进一步在全局变量表(即主符号表)中查找,若还没有找到,则说明引用的名字没有声明。

数组下标越界的错误可以在语义分析阶段或者程序执行阶段发现。例如在某C语言程序中声明数组变量: $\text{int } a[10]$,而在可执行语句中通过下标表达式引用数组元素。若下标表达式是一个整型常数,例如 11,则编译程序将发现引用 $a[11]$ 是错误的;若下标表达式是一个整型变量,例如 i ,若编译程序可以分析出 i 有值,并且 $i < 0$ 或者 $i > 9$,则引用 $a[i]$ 是错误的。即,若编译程序可以确定下标表达式的值,则数组下标越界的错误可以在语义分析时发现;如果编译程序无法确定下标表达式的值,则数组越界的错误只能在程序运行中出现存储溢出才被发现。例如,程序通过整型变量 i 引用 $a[i]$,该引用出现在一个循环结构中,编译时将无法确定 i 的值,如果越界,也只能在程序运行中出现存储溢出时才能发现。

本应为常数,但在数中出现了非数字字符,这类错误有些可以在词法分析阶段发现,如 2.3.4、2.3@4、3.4E.5 等都属于词法错误;而有些只能在语法分析阶段发现,如程序员误将语句 $x=234$;写成了 $x=2w4$; ,这样的错误只有在语法分析程序分析赋值语句时才可以发现。

问题：

在“代码优化”章节中我们学到，传统编译器高度依赖“启发式规则（Heuristics）”（即专家凭经验设计的规则）做决策，例如哪些变量放入寄存器、是否展开循环等。然而随着技术发展，机器学习（ML）正越来越多地被应用于编译过程。

请谈谈相比于传统的人工启发式规则，AI技术是如何提升编译器性能的？特别是在复杂的优化场景下（如选择最优的优化参数组合），AI驱动的方法具备哪些优势？

答案：

1. 突破人工规则的局限（From Rules to Data）：

- 传统启发式规则是基于“通用情况”设计的，难以涵盖所有代码模式和硬件特性，且维护难度大。
- AI模型可以通过学习海量的代码库和性能数据，自动发现那些人类专家难以察觉的优化规律，针对特定的代码特征做出更精准的决策（如更智能的寄存器分配）。

2. 高效的搜索空间探索（Auto-tuning）：

- 编译器的优化选项（Pass）组合成千上万，人工很难找到针对某个程序的“全局最优解”。
- AI算法（如机器学习或搜索算法）可以自动化地在巨大的搜索空间中进行探索，快速找到针对当前硬件和程序的最佳编译配置（Auto-tuning），实现远超通用配置（如-O3）的性能。

问题：

近年来，“软件供应链安全”成为全球关注焦点，攻击往往瞄准开源组件或构建过程本身。编译器作为软件构建的核心工具，是安全的“守门人”。

请谈谈你对编译器与软件安全关系的理解。除了翻译代码，编译器在确保我们发布的软件是“安全可信”的方面，还能发挥什么作用？

答案：

1. 代码审计与漏洞扫描：

- 编译器在编译过程中拥有代码的完整视图，可以进行深度的静态分析，自动发现潜在的内存泄露、越界访问等安全漏洞，将风险拦截在发布之前。

2. 可重现构建（Reproducible Builds）：

- 通过编译技术的改进，确保在不同环境、不同时间对同一源代码编译出的二进制文件完全一致，从而防止构建环境中被植入后门或恶意篡改，建立软件的信任链。
-

问题：

随着人工智能的飞速发展，AI编程助手已能生成高质量代码，甚至有观点认为“程序员将消失”。在此背景下，有同学可能会质疑学习“编译原理”这种底层理论的必要性。

请结合本课程的学习，反驳这一观点。谈谈在AI时代，理解编译器内部工作机制对于成为一名高水平的软件工程师为何依然至关重要？

答案：

1. 打破黑盒 (Deep Understanding)：

- AI生成的代码可能包含难以察觉的错误或性能瓶颈。只有理解编译原理（如语法分析、内存布局），工程师才能透过现象看本质，具备调试复杂问题和优化底层性能的能力。

2. 工具创造者 (Tool Building)：

- AI只是工具，而编译原理是制造工具的学科。无论是设计新的领域特定语言 (DSL)、优化AI推理引擎，还是构建静态分析工具，都离不开编译技术。懂原理才能驾驭AI，而不是被AI取代。
-

问题：

软件安全是国家网信事业的基石。统计表明，绝大多数系统漏洞源于内存安全问题。现代系统级编程语言（如Rust）因其在编译期强制保证安全而受到追捧。

- 请从“静态语义分析”的角度，解释编译器是如何检测出那些在旧语言中通常只在运行时才出现的错误（如缓冲区溢出、悬空指针）的？
- 为什么“左移”（将检查从运行时推前到编译时）被认为是编译器技术未来发展的一个主要趋势？

答案：

1. 静态分析原理：

- 数据流分析与类型系统：编译器不仅仅检查语法，还通过构建控制流图 (CFG) 进行深入的数据流分析，追踪变量的生命周期 (Liveness Analysis) 和所有权状态。
- 约束检查：通过引入更严格的类型系统（如借用检查 Borrow Checker），编译器在生成代码前就能推导出是否存在非法内存访问的可能性，从而报错。

2. 左移趋势的意义：

- 零成本抽象 (Zero-overhead)：如果在编译期就能证明代码是安全的，运行时就不需要额外的检查指令或垃圾回收 (GC) 机制，从而在保证安全的同时不牺牲性能。
 - 降低修复成本：软件工程规律表明，Bug发现得越早（在开发/编译阶段），修复成本越低，对系统的危害也越小。
-

问题：

国产芯片生态的成熟高度依赖于“软硬接口”，其中编译工具链起着决定性作用。目前，开源指令集RISC-V为我国半导体产业提供了历史性的机遇。结合课程中“目标代码生成”及编译器架构的知识：

1. 请分析为什么强大的“编译器后端”是新芯片架构（如国产RISC-V芯片）构建软件生态的关键？
2. 现代编译器的模块化设计（如LLVM的前端-IR-后端结构）是如何加速国产芯片适配现有海量软件资源的？

答案：

1. 编译器后端的关键作用：
 - 性能释放：芯片的理论性能（如流水线、多核）需要通过编译器后端的指令调度（Instruction Scheduling）和寄存器分配（Register Allocation）才能转化为实际算力。没有优秀的后端，芯片只能发挥出部分性能。
 - 生态准入：只有具备了稳定的编译器后端，操作系统（如Linux, OpenHarmony）和应用软件才能被编译运行在该芯片上，否则芯片无法运行主流软件。
 2. 模块化设计的优势：
 - 复用性与解耦合：采用公共的中间表示（IR），国产芯片厂商只需专注于开发针对自家硬件的“后端”模块，即可直接复用成熟的“前端”（如Clang支持C/C++），无需从头开发整个编译器。
 - 快速迁移：这使得现有的海量软件资产可以通过“重新编译”低成本地迁移到国产硬件上，解决了新架构面临的“应用匮乏”难题。
-

问题：

当前趋势显示人工智能与编译技术正在深度融合。随着大模型（LLMs）的广泛应用以及异构计算（如NPU、GPU）的兴起，传统“以CPU为中心”的编译模式正受到挑战。结合课程中“中间表示（IR）”和“代码优化”的知识：

1. 为什么现代深度学习编译器（如TVM, MLIR）需要在低级IR（指令级）之外引入高级IR（张量级）？
2. 在AI生成代码的时代，你认为编译器的角色将如何转变，以保障代码的正确性与性能？

答案：

1. 关于高级IR的必要性：
 - 保留语义信息：传统的低级IR（如LLVM IR）过于贴近硬件指令，容易丢失高层的数学语义（如矩阵乘法、卷积）。引入张量级（Tensor-level）IR可以保留这些信息，便于进行特定领域的算法优化（如算子融合 Operator Fusion）。

- 适应异构硬件：不同的AI加速器（GPU/NPU）有不同的存储结构和计算单元，高级IR更容易映射到这些复杂的硬件原语上，而不仅是线性的指令序列。

2. 编译器的角色转变：

- 从“翻译”到“验证”：随着AI生成代码的增多，编译器将更多承担“守门员”的角色，利用静态分析技术快速检测AI生成代码中的逻辑漏洞和安全隐患。
 - 智能化优化：编译器自身将集成AI模型（AI for Compiler），自动寻找最优的编译参数和调度策略（Auto-tuning），取代人工编写的启发式规则。
-

问题：符号表（Symbol Table）的主要作用及具体操作。

答案：

1. 词法分析与语法分析阶段（Lexical & Syntax Analysis）

- 作用：识别源代码中的标识符（变量名、函数名等），并在处理声明语句时建立初步的符号表条目。
- 具体操作：
 - Insert（插入）：当扫描到新的标识符名称或解析到声明语句（如 `int a;`）时，将“a”及其基本信息加入符号表。
 - Lookup（查找）：检查该标识符是否为保留字（关键字），或者在同一作用域内是否重复声明。

2. 语义分析阶段（Semantic Analysis）

- 作用：这是符号表使用最频繁的阶段。编译器利用符号表进行类型检查（Type Checking）和作用域（Scope）验证，确保变量先声明后使用，且类型匹配。
- 具体操作：
 - Lookup（查找）：当代码中引用一个变量（如 `x = y + 1`）时，查找 `x` 和 `y` 是否已定义，并获取它们的数据类型以检查运算是否合法。
 - Update（更新/设置）：补充完善符号的属性，例如将推导出的数据类型、参数列表信息填入表项中。
 - Scope Management（作用域管理）：进入或退出代码块时，执行创建子表或删除/隐藏局部变量的操作。

3. 中间代码与目标代码生成阶段（Code Generation）

- 作用：辅助地址分配。编译器需要知道每个变量占用的空间大小及其在内存中的相对位置（偏移量），以便生成正确的指令。
- 具体操作：
 - Update（更新）：计算并记录变量在活动记录（Activation Record）中的相对地址（Offset）。

- **Lookup (查找)**：在生成汇编或机器码时，从表中读取变量的地址/偏移量，将其转换为具体的内存地址指令。